

Міністерство освіти та науки України

Національний університет “Львівська політехніка”

Інститут інформаційно-комунікаційних технологій та електронної
інженерії



ЗВІТ

З Лабораторної роботи 2
з дисципліни “Об’єктно-орієнтовне програмування частина 2”

Виконав:

Ст. гр. ІХ-22

Ненадовський Р.М.

Прийняв:

Плесканка М.В.

Львів 2026

Тема: Моделювання телекомунікаційної мережі з використанням генераторів подій, протоколів TCP/UDP, асинхронного моделювання та візуалізація продуктивності.

Мета: Ознайомлення з основами моделювання мереж (зірка, кільце, сітка), вивчення використання ООП для створення об'єктів вузлів і маршрутизаторів, а також аналіз затримок та втрат пакетів.

Завдання:

Розробити модель телекомунікаційної мережі з використанням ООП.

Основні вимоги:

- реалізувати класи вузлів, пакетів та з'єднань;
- змодельовати передачу даних через TCP та UDP;
- використати генератори подій або asyncio;
- підтримати різні топології мережі.

Додатково:

- реалізувати збір статистики (затримка, втрати);
- створити візуалізацію мережі.

Код:

```
import asyncio
import random
import time
import networkx as nx
import matplotlib.pyplot as plt

#Завдання 1.(КЛАС ВУЗЛА МЕРЕЖІ)
class Node: 9 usages  ⚡ Роман Ненадовський

    def __init__(self, name):  ⚡ Роман Ненадовський
        self.name = name
        self.connections = []

    def connect(self, node): 2 usages  ⚡ Роман Ненадовський
        self.connections.append(node)
        node.connections.append(self)
# -----

#Завдання 2.(КЛАС МАРШРУТИЗАТОРА)
class Router(Node): 1 usage  ⚡ Роман Ненадовський
    pass
# -----

#Завдання 3.(КЛАС ПАКЕТА)
class Packet: 2 usages  ⚡ Роман Ненадовський

    def __init__(self, src, dest, size, protocol):  ⚡ Роман Ненадовський
        self.src = src
        self.dest = dest
        self.size = size
        self.protocol = protocol
# -----
```

```

#Завдання 4.(TCP ПРОТОКОЛ)
class TCPProtocol: 1 usage  ⚡ Роман Ненадовський

    name = "TCP"

    @staticmethod 1 usage (1 dynamic)  ⚡ Роман Ненадовський
    async def transmit(src, dest, network):

        packet = Packet(src, dest, random.randint(a=200, b=500), protocol="TCP")

        await asyncio.sleep(random.uniform(a=0.05, b=0.2))

        if random.random() < network.loss_rate:
            network.packets_lost += 1
            print("TCP packet lost")
        else:
            print(f"TCP packet delivered {src.name} -> {dest.name}")
# -----

```

```

#Завдання 5.UDP ПРОТОКОЛ
class UDPProtocol: 1 usage  ⚡ Роман Ненадовський

    name = "UDP"

    @staticmethod 1 usage (1 dynamic)  ⚡ Роман Ненадовський
    async def transmit(src, dest, network):

        packet = Packet(src, dest, random.randint(a=50, b=200), protocol="UDP")

        await asyncio.sleep(random.uniform(a=0.05, b=0.2))

        if random.random() < network.loss_rate:
            network.packets_lost += 1
            print("UDP packet lost")
        else:
            print(f"UDP packet delivered {src.name} -> {dest.name}")

```

```

#Завдання 6.(КЛАС МЕРЕЖІ)
# -----
class Network:

    def __init__(self):

        self.nodes = []

        self.loss_rate = random.uniform(0.10, 0.15)

        self.packets_sent = 0
        self.packets_lost = 0

        self.total_time = 0

    async def simulate(self, protocol, count=10):

        for i in range(count):

            src, dest = random.sample(self.nodes, 2)

            start = time.time()

            self.packets_sent += 1

            await protocol.transmit(src, dest, self)

            end = time.time()

            self.total_time += (end - start)

```

```

def analyze(self):

    avg_time = self.total_time / self.packets_sent

    loss_percent = (self.packets_lost / self.packets_sent) * 100

    bandwidth = (self.packets_sent - self.packets_lost) / self.total_time

    print("\n--- Network statistics ---")

    print("Average time:", round(avg_time, 4), "sec")

    print("Packet loss:", round(loss_percent, 2), "%")

    print("Bandwidth:", round(bandwidth, 2), "packets/sec")

def visualize(self, title):

    G = nx.Graph()

    for node in self.nodes:
        for conn in node.connections:
            G.add_edge(node.name, conn.name)

    nx.draw(G,
            with_labels=True,
            node_color="lightblue",
            node_size=2500)

    plt.title(title)

    plt.show()

```

```

#Завдання 7. (ЗІРКОВА ТОПОЛОГІЯ)
✓ async def star_topology():

    network = Network()

    router = Router("Router")

    pcs = [Node("PC1"), Node("PC2"), Node("PC3"), Node("PC4")]

    network.nodes = [router] + pcs

    for pc in pcs:
        router.connect(pc)

    await network.simulate(TCPProtocol)

    network.analyze()

    network.visualize("Star topology")
    # -----

#Завдання 8. (СІТКОВА ТОПОЛОГІЯ)
✓ async def mesh_topology():

    network = Network()

    nodes = [Node("PC1"), Node("PC2"), Node("PC3"), Node("PC4")]

    network.nodes = nodes

    for i in range(len(nodes)):
        for j in range(i + 1, len(nodes)):
            nodes[i].connect(nodes[j])

    await network.simulate(UDPProtocol)

    network.analyze()

    network.visualize("Mesh topology")

```

```
#Завдання 9.(ГОЛОВНА ФУНКЦІЯ)

async def main():

    print("STAR NETWORK")

    await star_topology()

    print("\nMESH NETWORK")

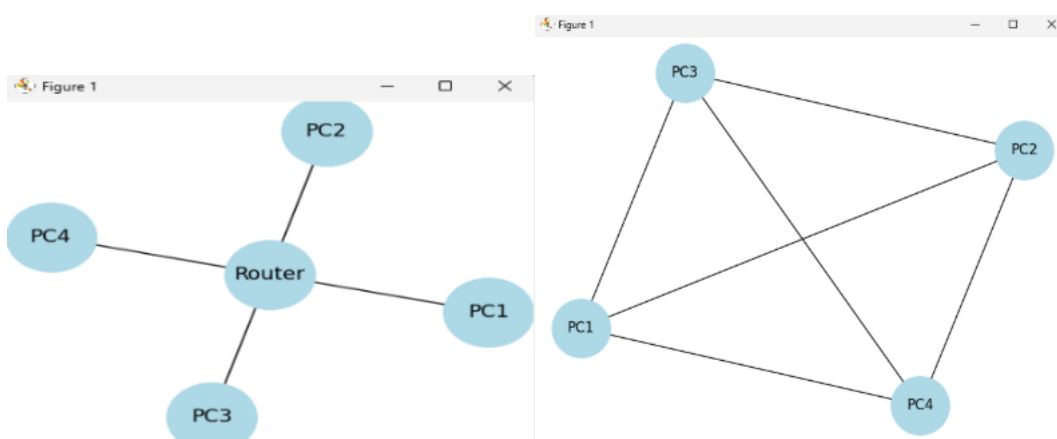
    await mesh_topology()

asyncio.run(main())
```

Результат:

Результат:

STAR NETWORK	MESH NETWORK
TCP packet delivered PC1 -> PC2	UDP packet delivered PC1 -> PC4
TCP packet delivered Router -> PC1	UDP packet delivered PC1 -> PC4
TCP packet delivered PC1 -> PC2	UDP packet delivered PC3 -> PC2
TCP packet lost	UDP packet delivered PC2 -> PC1
TCP packet delivered Router -> PC1	UDP packet delivered PC4 -> PC3
TCP packet delivered PC3 -> PC4	UDP packet delivered PC4 -> PC2
TCP packet delivered Router -> PC4	UDP packet delivered PC3 -> PC1
TCP packet delivered PC3 -> PC4	UDP packet delivered PC3 -> PC1
TCP packet delivered PC1 -> PC4	UDP packet delivered PC1 -> PC3
TCP packet delivered PC2 -> Router	UDP packet delivered PC1 -> PC2
---	---
Network statistics ---	Network statistics ---
Average time: 0.1215 sec	Average time: 0.1468 sec
Packet loss: 10.0 %	Packet loss: 0.0 %
Bandwidth: 7.41 packets/sec	Bandwidth: 6.81 packets/sec



Висновок: Використання об'єктно-орієнтованого підходу дозволяє ефективно імітувати роботу реальних мережевих пристроїв, а застосування бібліотеки `networkx` забезпечує наочну візуалізацію топологій для порівняльного аналізу їхньої продуктивності.

